

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE CODE:** MLA03

**COURSE NAME:** Reinforcement learning for Environment and Agent

**NAME:** M.Madhulika

**Register Number:**192472188

---

### COURSE OUTCOME COVERED

*CO3: Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

*Assessment Tool Weightage: 30%*

---

Dr G. A. SENTHIL

**QUESTIONS: 10**

**Total Marks: 50**

Annexure A

### CODING CHALLENGE

**1. Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making.**

**Solution: -**

#### **Problem Statement**

Design a Reinforcement Learning (RL) framework where an intelligent agent interacts with an environment, learns from rewards, and selects actions to maximize long-term performance.

#### **Objectives**

- Understand the RL framework.
- Explain agent–environment interaction.
- Define states, actions, and rewards.
- Show how cumulative reward improves decision-making.

#### **Programming Language**

- Python 3.x

## Environment Setup

### Software

- Python 3.x
- Google Collab / Jupyter Notebook / VS Code

### Libraries

- NumPy
- Matplotlib

### Input

The RL agent receives:

- Current state
- Available actions
- Reward from the environment

### Output

The program displays:

- Selected action
- Reward received
- Updated policy
- Total cumulative reward

### Program

```
import numpy as np
states = ["S1", "S2", "Goal"]
actions = ["Left", "Right"]
current_state = "S1"
total_reward = 0
print("Initial State:", current_state)
for i in range(5):
    action = np.random.choice(actions)
    if action == "Right":
        reward = 10
        current_state = "Goal"
    else:
        reward = -2
        current_state = "S1"
    total_reward += reward
    print("Action:", action)
    print("Reward:", reward)
    print("State:", current_state)
    print("Cumulative Reward:", total_reward)
    print()
```

**Sample Output: -**

Initial State: S1  
Action: Right  
Reward: 10  
State: Goal  
Cumulative Reward: 10  
Action: Left  
Reward: -2  
State: S1  
Cumulative Reward: 8

**Conclusion**

Reinforcement Learning enables an agent to learn optimal decisions through continuous interaction with the environment. By observing states, taking actions, and receiving rewards, the agent improves its policy over time and maximizes cumulative rewards, leading to better decision-making.

**2. Estimate how decision-making problems can be modelled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor.****Solution: -****Problem Statement**

Design a Markov Decision Process (MDP) model for a decision-making problem where an agent selects actions based on the current state to maximize long-term rewards.

**Objectives**

- Understand the Markov Decision Process (MDP).
- Define the components of an MDP.
- Explain action space, transition probability, reward function, and discount factor.
- Show how MDP helps in sequential decision-making.

**Programming Language**

- Python 3.x

**Environment Setup****Software**

- Python 3.x
- Google Collab / Jupyter Notebook / VS Code

**Libraries**

- NumPy
- Matplotlib

## Input

The MDP receives:

- Current state
- Available actions
- Transition probabilities
- Reward values
- Discount factor ( $\gamma$ )

## Output

The program displays:

- Selected action
- Next state
- Reward obtained
- Updated value or policy

## MDP Components

- State (S): Current situation of the agent.
- Action (A): Possible actions the agent can perform.
- Transition Probability (P): Probability of moving to the next state after an action.
- Reward Function (R): Immediate reward received after taking an action.
- Discount Factor ( $\gamma$ ): Determines the importance of future rewards.

## Program-

```
import numpy as np

states = ["S1", "S2", "Goal"]

actions = ["Right", "Left"]

current_state = "S1"

gamma = 0.9

for i in range(5):

    action = np.random.choice(actions)

    if action == "Right":

        next_state = "S2"

        reward = 10

    else:

        next_state = "S1"
```

```
reward = -2

value = reward * gamma

print("Current State:", current_state)

print("Action:", action)

print("Next State:", next_state)

print("Reward:", reward)

print("Updated Value:", value)

print()

current_state = next_state
```

### **Sample output-**

Current State: S1

Action: Right

Next State: S2

Reward: 10

Updated Value: 9.0

Current State: S2

Action: Left

Next State: S1

Reward: -2

Updated Value: -1.8

### **Conclusion**

A Markov Decision Process (MDP) provides a structured framework for solving sequential decision-making problems. By defining states, actions, transition probabilities, rewards, and the discount factor, an agent can learn an optimal policy that maximizes long-term rewards and improves decision-making over time.

### **3. Justify the exploration–exploitation trade-off in RL and compare strategies such as $\epsilon$ -greedy and SoftMax approaches in balancing learning efficiency.**

**Solution: -**

#### **Problem Statement**

Design a Reinforcement Learning (RL) system that balances exploration (trying new actions) and exploitation (using the best-known actions) to achieve maximum learning efficiency and long-term rewards.

#### **Objectives**

- Understand the exploration–exploitation trade-off.
- Compare  $\epsilon$ -greedy and SoftMax strategies.
- Improve learning efficiency through balanced decision-making.
- Maximize cumulative rewards.

#### **Programming Language**

- Python 3.x

#### **Environment Setup**

##### **Software**

- Python 3.x
- Google Collab / Jupyter Notebook / VS Code

##### **Libraries**

- NumPy
- Matplotlib

##### **Input**

The RL agent receives:

- Current state
- Available actions
- Action values (Q-values)
- Reward from the environment

## Output

The program displays:

- Selected action
- Reward received
- Updated Q-values
- Best policy after learning

## Comparison of Strategies

| $\epsilon$ -Greedy                              | SoftMax                                     |
|-------------------------------------------------|---------------------------------------------|
| Chooses the best action most of the time.       | Selects actions based on probability.       |
| Explores randomly with probability $\epsilon$ . | Gives higher probability to better actions. |
| Simple and easy to implement.                   | More balanced exploration.                  |
| Faster learning but may ignore some actions.    | Better for complex environments.            |

## Program-

```
import numpy as np

actions = ["Left", "Right", "Up", "Down"]
q_values = np.array([2.5, 4.0, 1.5, 3.0])
epsilon = 0.2

if np.random.rand() < epsilon:
    action = np.random.choice(actions)
    method = "Exploration ( $\epsilon$ -Greedy)"
else:
    action = actions[np.argmax(q_values)]
```

```
method = "Exploitation ( $\epsilon$ -Greedy)"
reward = np.random.randint(1, 11)
q_values[actions.index(action)] += 0.1 * reward
temperature = 1.0
prob = np.exp(q_values / temperature)
prob = prob / np.sum(prob)
softmax_action = np.random.choice(actions, p=prob)
print("Method:", method)
print("Selected Action:", action)
print("Reward:", reward)
print("Updated Q-values:", q_values)
print("SoftMax Action:", softmax_action)
```

### **Sample Output-**

Method: Exploitation ( $\epsilon$ -Greedy)

Selected Action: Right

Reward: 8

Updated Q-values: [2.5 4.8 1.5 3.0]

SoftMax Action: Right

### **Conclusion**

The exploration–exploitation trade-off is essential in Reinforcement Learning. The  $\epsilon$ -greedy strategy is simple and effective for many problems, while the softmax strategy provides a more balanced approach by selecting actions based on probabilities. By balancing exploration and exploitation, the RL agent learns an optimal policy and maximizes long-term rewards.

## **4. Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behaviour.**

**Solution: -**

**Problem Statement**

Develop a Reinforcement Learning (RL) model using the Q-learning algorithm to enable an agent to learn the best action in each state by updating Q-values through repeated interactions with the environment.

### **Objectives**

- Understand the Q-learning algorithm.
- Learn how Q-values are updated.
- Find the optimal policy through trial and error.
- Maximize cumulative rewards.

### **Programming Language**

- Python 3.x

### **Environment Setup**

#### **Software**

- Google Collab / Jupyter Notebook / VS Code

#### **Libraries**

- NumPy
- Matplotlib

### **Input**

The RL agent receives:

- Current state
- Available actions
- Reward from the environment
- Next state

### **Output**

The program displays:

- Updated Q-values
- Selected action
- Reward received
- Final optimal policy

### **Python Code-**

```
import numpy as np

Q = np.zeros((2,2))

alpha = 0.1

gamma = 0.9

state = 0

action = 1

reward = 10

next_state = 1

Q[state, action] = Q[state, action] + alpha * (reward + gamma * np.max(Q[next_state]) - Q[state, action])

print("Updated Q-Table:")

print(Q)

policy = np.argmax(Q, axis=1)

print("Optimal Policy:", policy)
```

### **Sample Output-**

Updated Q-Table:

```
[[0. 1.]
```

```
[0. 0.]]
```

Optimal Policy:

```
[1 0]
```

### **Conclusion**

Q-learning enables an RL agent to learn the best action for every state by continuously updating Q-values based on rewards. As training progresses, the Q-values converge toward optimal values, allowing the agent to make intelligent decisions and maximize long-term rewards.

**5. Differentiate between on-policy and off-policy learning methods in RL, and assess their advantages and limitations in practical applications.**

## **Solution: -**

### **Problem Statement**

Design a Reinforcement Learning (RL) framework to compare on-policy and off-policy learning methods. Analyze how each method learns from the environment and evaluate their advantages and limitations.

### **Objectives**

- Understand on-policy and off-policy learning.
- Compare their learning approaches.
- Identify advantages and limitations.
- Select the appropriate method for different RL problems.

### **Programming Language**

- Python 3.x

### **Libraries**

- NumPy
- Matplotlib

### **Input**

The RL agent receives:

- Current state
- Selected action
- Reward from the environment
- Next state

### **Output**

The program displays:

- Selected policy
- Reward received
- Updated action values
- Final learned policy

## Comparison of Learning Methods

| On-Policy Learning                    | Off-Policy Learning                          |
|---------------------------------------|----------------------------------------------|
| Learns using the current policy.      | Learns using a different target policy.      |
| Continuously updates the same policy. | Uses stored experiences to improve learning. |
| More stable and safer.                | More sample-efficient and faster.            |
| Example: SARSA                        | Example: Q-Learning                          |

### Program-

```
import numpy as np

Q = np.zeros((2,2))

alpha = 0.1

gamma = 0.9

state = 0

action = 1

reward = 10

next_state = 1

next_action = 0

Q[state, action] += alpha * (reward + gamma * Q[next_state, next_action] - Q[state, action])

print("SARSA (On-Policy) Q-Table:")

print(Q)

Q = np.zeros((2,2))

Q[state, action] += alpha * (reward + gamma * np.max(Q[next_state]) - Q[state, action])

print("\nQ-Learning (Off-Policy) Q-Table:")

print(Q)
```

```
print("\nOptimal Policy:", np.argmax(Q, axis=1))
```

### **Sample Output-**

SARSA (On-Policy) Q-Table:

```
[[0. 1.]
```

```
[0. 0.]]
```

Q-Learning (Off-Policy) Q-Table:

```
[[0. 1.]
```

```
[0. 0.]]
```

Optimal Policy:

```
[1 0]
```

### **Conclusion**

On-policy and off-policy learning are two important approaches in Reinforcement Learning. On-policy methods, such as SARSA, learn using the current policy and provide stable learning, while off-policy methods, such as Q-Learning, learn from a target policy and often achieve faster learning. Choosing the appropriate method depends on the application, learning speed, and environment.

### ***Code of Conduct:***

*I M.Madhulika (192472188) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

### ***Signature of the student:***

**M.Madhulika**

## RUBRICS FOR CALCULATION

| Criteria              | Weightage | Level 4 (Exemplary)                                                               | Level 3 (Proficient)                             | Level 2 (Developing)                               | Level 1 (Beginning)                   |
|-----------------------|-----------|-----------------------------------------------------------------------------------|--------------------------------------------------|----------------------------------------------------|---------------------------------------|
| Technical Knowledge   | 25        | Demonstrates strong and accurate subject knowledge with in-depth understanding    | Shows good knowledge with minor gaps             | Basic knowledge with noticeable errors             | Lacks fundamental technical knowledge |
| Problem Solving       | 25        | Solves problems logically and applies concepts effectively in real scenarios      | Applies concepts with moderate accuracy          | Limited problem-solving ability; requires guidance | Unable to solve or apply concepts     |
| Analytical Thinking   | 15        | Analyses questions critically and provides well-structured, logical answers       | Provides reasonable analysis with some structure | Superficial or partially structured responses      | No analytical thinking demonstrated   |
| Communication         | 15        | Communicates clearly, confidently, and professionally with proper terminology     | Communicates well with minor hesitation          | Communication lacks clarity or confidence          | Unable to communicate effectively     |
| Confidence            | 15        | Displays high confidence, positive attitude, and professional behavior throughout | Shows good confidence with minor issues          | Low confidence; inconsistent behavior              | Lacks confidence and professionalism  |
| Response to Questions | 10        | Responds accurately, promptly, and elaborates with relevant examples              | Responds correctly but with limited depth        | Partial responses; needs prompting                 | Unable to respond appropriately       |